

第5章 C++与STL入门

学习目标

- ☑ 熟悉 C++ 版算法竞赛程序框架
- ☑ 理解变量引用的原理
- ☑ 熟练掌握 string 与 stringstream
- ☑ 熟练掌握 C++ 结构体的定义和使用，包括构造函数和静态成员变量
- ☑ 了解常见的可重载运算符，包括四则运算、赋值、流式输入输出、()和[]
- ☑ 了解模板函数和模板类的概念
- ☑ 熟练掌握 STL 中排序和检索的相关函数
- ☑ 熟练掌握 STL 中 vector、set 和 map 这 3 个容器
- ☑ 了解 STL 中的集合相关函数
- ☑ 理解栈、队列和优先队列的概念，并能用 STL 实现它们
- ☑ 熟练掌握随机数生成方法，并能结合 assert 宏进行测试
- ☑ 能独立编写大整数类 BigInteger

在前 4 章中介绍了 C 语言的主要内容，已经足以应付许多算法竞赛的题目了。然而，“能写”并不代表“好写”，有些题目虽然可以用 C 语言写出来，但是用 C++ 写起来往往会更快，而且更不容易出错，所以在讨论算法之前，有必要对 C++ 进行一番讲解。

本章采用“实用主义”的写法，并不会对所有内容加以解释，但是这并不影响读者“依葫芦画瓢”。不过有时读者还是希望能更细致、准确地学习到相关知识。推荐读者在手边放一本 C++ 的参考读物，如 C++ 之父 Bjarne Stroustrup 的经典著作《C++ 程序设计语言》。尽管如此，本章的作用也不容忽视：C++ 是一门庞大的语言，大多数语言特性和库函数在算法竞赛中都是用不到（或者可以避开）的。而且算法竞赛有它自身的特点，即使对于资深 C++ 程序员来说，如果缺乏算法竞赛的经验，也很难总结出一套适用于算法竞赛的知识点和实践指南。因此，即使你已经很熟悉 C++ 语言，但笔者仍建议花一些时间浏览本章的内容，相信会有新的收获。

5.1 从 C 到 C++

C 语言是一门很有用的语言，但在算法竞赛中却不流行，原因在于它太底层，缺少一些“实用的东西”。例如，在 2013 年 ACM/ICPC 世界总决赛中，有 1347 份用 C++ 提交，323 份用 Java 提交，但一份用 C 提交的都没有。

既然如此，为什么还要花这么多篇幅介绍 C 语言呢？答案是 C++ 太复杂了。与其把 C++

学得一知半解，还不如先把 C 语言的基础打好。前面已经提到过，前 4 章的所有代码都可以直接作为 C++ 程序进行编译，所以请把前 4 章内容看作语言的核心部分，而把本章内容看作是可选的工具。如果某些工具难以掌握，索性避开就是了。

C++ 博大精深，但也有很多人诟病的地方。好在算法竞赛中的大多数选手只会用到其中很少的特性，本章的任务就是把这些特性介绍给读者，以供选用。

提示 5-1: C++ 的精华与糟粕并存。本章介绍的 C++ 特性是算法竞赛中最常用的部分，虽然不是解题所必需的，但值得学习。

5.1.1 C++ 版框架

虽然前面介绍的内容都可以直接用在 C++ 程序里，但有些并不是 C++ 的推荐写法，只是为了更好地兼容 C 语言才如此编写的。下面是 C++ 版的“a+b 程序”：

```
#include<cstdio>
int main() {
    int a, b;
    while(scanf("%d%d", &a, &b) == 2) printf("%d\n", a+b);
    return 0;
}
```

和之前的 C 程序比较，唯一的区别是 `stdio.h` 变成了 `cstdio`。事实上，`stdio.h` 仍然存在，但是 C++ 中推荐的头文件是 `cstdio`。类似地，`string.h` 变成了 `cstring`，`math.h` 变成了 `cmath`，`ctype.h` 变成了 `cctype`。带 `.h` 后缀的头文件依然存在，但并不被 C++ 所推荐使用。

提示 5-2: C++ 能编译大多数 C 语言程序。虽然 C 语言中大多数头文件在 C++ 中仍然可以使用，但推荐的方法是在 C 头文件之前加一个小写的 `c` 字母，然后去掉 `.h` 后缀。

下面是一个稍微复杂一点的程序，它展示了更多的常用 C++ 特性。

```
#include<iostream>
#include<algorithm>
using namespace std;
const int maxn = 100 + 10;
int A[maxn];
int main() {
    long long a, b;
    while(cin >> a >> b) {
        cout << min(a,b) << "\n";
    }
    return 0;
}
```

这次的变化就大多了，新增的两个头文件不再是以字符 `c` 开头的。有人会猜这一定是 C++ 特有的头文件——的确如此。`iostream` 提供了输入输出流，而 `algorithm` 提供了一些常用

算法，例如代码中的 `min`^①。`cin >> a` 的含义是从标注输入中读取 `a`，它的返回值是一个“已经读取了 `a` 的新流”，然后从这个新流中继续读取 `b`。如果流已经读完，`while` 循环将退出。这种方式相比 `scanf` 的最大优势就是不再需要记忆 `%d`、`%s` 等占位符，同时也避开了前面提到的“`long long` 类型的输入输出占位符不统一”的问题。当然，C++流也不是完美的，其最大缺点就是运行太慢，以至于很多竞赛题目会在题面中的显著位置注明：本题的输入量很大，请不要使用 C++ 的流输入^②。

还有一个新内容：`using namespace std`。这是什么意思呢？C++ 中有一个“名称空间”（`namespace`）的概念，用来缓解复杂程序的组织问题。例如张三写了一个函数叫 `my_good_function`（意思是“我的优秀函数”），李四也写了这样一个函数，但作用和张三的不同。如果有一天需要把他们的程序合在一起用，就会出问题：函数不能重名。虽然后面会讲到 C++ 支持函数重载，但如果这两个函数的参数类型也完全相同，则是不能重载的。一个解决方案是分别把函数写在各自的名称空间里，然后就可以用 `zhang3:my_good_function()` 和 `li4:my_good_function()` 这样的方式进行调用了。

基于这样的考虑，头文件 `iostream` 和 `algorithm` 里定义的内容放在 `std` 名称空间里。如果代码和该名称空间里的内容不重名，就可以用 `using namespace std` 的方法把 `std` 里的名字导入默认空间^③。这样就可以用 `cin` 代替 `std::cin`，`cout` 代替 `std::cout`，`min` 代替 `std::min` 了。不信的话，你可以把这行语句注释掉，再编译一次试试。

提示 5-3: C++ 中可以使用流简化输入输出操作。标准输入输出流在头文件 `iostream` 中定义，存在于名称空间 `std` 中。如果使用了 `using namespace std` 语句，则可以直接使用。

最后还有一个细节：声明数组时，数组大小可以使用 `const` 声明的常数（这在 C99 中是不允许的）。在 C++ 中，这种写法更为推荐，因此本书后面的代码中一律采用这样的写法，而不是用 `#define` 声明常数。

顺便一提，C++ 中的数据类型和 C 语言很接近，最显著的区别是多了一个 `bool` 来表示布尔值，然后用 `true` 和 `false` 分别表示真和假。虽然仍然可以用 `int` 来表示真假，但是用 `bool` 可以让程序更清晰。

5.1.2 引用

第 4 章中曾经介绍过交换两个变量的方法，最后给出的例子用到了指针，看上去不太自然。C++ 提供了“引用”，虽然在功能上比指针弱，但是减少了出错的可能，提高了代码的可读性。使用引用交换两个变量的方法如下：

```
#include<iostream>
using namespace std;

void swap2(int& a, int& b) {
```

^① C 语言里连 `min` 函数都没有，可想而知还有多少常用的东西是无法直接用的。

^② 不过流也可以加速，方法是关闭和 `stdio` 的同步，即调用 `ios::sync_with_stdio(false)`。

^③ 在工程上不推荐这样做，不过因为算法竞赛的程序通常很小（多数不到 200 行），所以这样做也无大碍。

```
int t = a; a = b; b = t;
}
```

```
int main() {
    int a = 3, b = 4;
    swap2(a, b);
    cout << a << " " << b << "\n";
    return 0;
}
```

是不是很自然？如果在参数名之前加一个“&”符号，就表示这个参数按照传引用（by reference）的方式传递，而不是C语言里的传值（by value）方式传递。这样，在函数内改变参数的值，也会修改到函数的实参。按照第4章介绍的方法进行gdb调试，用**b swap2**加一个端点，然后用**r**命令执行，如下所示：

```
Breakpoint 1, swap2 (a=@0x22ff4c: 3, b=@0x22ff48: 4) at swap2.cpp:5
```

```
5 int t = a; a = b; b = t;
```

```
(gdb) bt
```

```
#0 swap2 (a=@0x22ff4c: 3, b=@0x22ff48: 4) at swap2.cpp:5
```

```
#1 0x004013e1 in main() at swap2.cpp:10
```

```
(gdb) up
```

```
#1 0x004013e1 in main() at swap2.cpp:10
```

```
10 swap2(a, b);
```

```
(gdb) print &a
```

```
$1 = (int *) 0x22ff4c
```

```
(gdb) print &b
```

```
$2 = (int *) 0x22ff48
```

看到了吗？main函数里的变量a、b的地址和swap2执行时参数a、b引用的地址一样，实际上是“同一个东西”。

提示 5-4：C++中的引用就是变量的“别名”，它可以在一定程度上代替C中的指针。例如，可以用“传引用”的方式让函数内直接修改实参。

细心的读者可能注意到了，为什么函数叫swap2而不是swap呢？因为algorithm这个头文件里已经提供过了swap，可以直接使用。这个swap比此处所写的swap2强太多了：它不仅同时支持int、double等所有内置类型，甚至还支持用户自己编写的结构体。它是做到这点的呢？我们很快就要学习到。

5.1.3 字符串

还记得前面所说的“数组不是一等公民”吗？C语言中的字符串就是字符数组，所以也不是“一等公民”，处处受限。例如，如何编写一个函数，把两个字符串拼接成一个长字

字符串？这个任务看上去简单，实际上却暗藏陷阱：新字符串的存储空间从哪里来？从第4章最后的讨论中可以知道：不能在函数中定义一个数组然后返回它的地址，因为函数返回后其中局部变量的地址便失效了。因此“字符串拼接”函数必须申请新的内存空间以存放结果，用完之后还要将申请的空间“退回去”，这会很麻烦。另外，字符串数组本身并不保存字符串长度，每次需要时都要用 `strlen` 函数重算一次。如果字符串很长，则 `strlen` 函数的开销将不容忽视^①。为了避免不必要的 `strlen` 调用，可以在某个变量中保存字符串的长度，但这样一来，程序会变得更加复杂，难以调试。总而言之，C 语言处理字符串并不方便。

C++ 提供了一个新的 `string` 类型，用来替代 C 语言中的字符数组。用户仍然可以继续用字符数组当字符串用，但是如果希望程序更加简单、自然，`string` 类型往往是更好的选择。例如，C++ 的 `cin/cout` 可以直接读写 `string` 类型，却不能读写字符数组；`string` 类型还可以像整数那样“相加”，而在 C 语言里只能使用 `strcat` 函数。

提示 5-5：C++ 在 `string` 头文件里定义了 `string` 类型，直接支持流式读写。`string` 有很多方便的函数和运算符，但速度有些慢。

考虑这样一个题目：输入数据的每行包含若干个（至少一个）以空格隔开的整数，输出每行中所有整数之和。如果只能使用字符与字符数组，一般有两种方案：一是使用 `getchar()` 边读边算，代码较短，但容易写错，并且相对较难理解^②；二是每次读取一行，然后再扫描该行的字符，同时计算结果。如果使用 C++，代码可以很简单。

```
#include<iostream>
#include<string>
#include<sstream>
using namespace std;

int main() {
    string line;
    while(getline(cin, line)) {
        int sum = 0, x;
        stringstream ss(line);
        while(ss >> x) sum += x;
        cout << sum << "\n";
    }
    return 0;
}
```

`string` 类在 `string` 头文件中，而 `stringstream` 在 `sstream` 头文件中。首先用 `getline` 函数读一行数据（相当于 C 语言中的 `fgets`，但由于使用 `string` 类，无须指定字符串的最大长度），然后用这一行创建一个“字符串流”——`ss`。接下来只需像读取 `cin` 那样读取 `ss` 即可。

^① 如果已完成了第3章的思考题，相信对此深有感触。

^② 有些选手非常习惯这种思维方式，但是根据笔者的经验，也有很多选手非常不习惯这种思维方式。

提示 5-6: 可以把 string 作为流进行读写, 定义在 sstream 头文件中。

虽然 string 和 sstream 都很方便, 但 string 很慢, sstream 更慢, 应谨慎使用^①。

5.1.4 再谈结构体

C++除了支持结构体 struct 之外, 还支持类 class。C++不再需要用 typedef 的方式定义一个 struct, 而且在 struct 里除了可以有变量(称为成员变量)之外还可以有函数(称为成员函数)。在工程中, 一般用 struct 定义“纯数据”的类型, 只包含较少的辅助成员函数, 而用 class 定义“拥有复杂行为”的类型, 不过为了简单起见, 本书中只使用 struct 而不使用 class。另外, “成员变量”、“成员函数”、“构造函数”等很多 C++ struct 里新加的概念同样适用于 class^②, 所以不用担心在本章中学到的内容为“非主流”。

提示 5-7: C++中的结构体除了可以拥有成员变量(用 a.x 的方式访问)之外, 还可以拥有成员函数(用 a.add(1,2)的方式访问)。为了简单起见, 本书中只使用 struct 而不使用 class, 但 struct 的很多概念和写法同样适用于 class。

下面是一个例子:

```
#include<iostream>
using namespace std;

struct Point {
    int x, y;
    Point(int x=0, int y=0):x(x),y(y) {}
};

Point operator + (const Point& A, const Point& B) {
    return Point(A.x+B.x, A.y+B.y);
}

ostream& operator << (ostream &out, const Point& p) {
    out << "(" << p.x << ", " << p.y << ")";
    return out;
}

int main() {
    Point a, b(1,2);
    a.x = 3;
    cout << a+b << "\n";
}
```

^① 具体有多慢? 试试就知道了。请读者自行编写程序测试。

^② 事实上, 在 C++中 struct 和 class 最主要的区别是默认访问权限和继承方式不同, 而其他方面的差异很小。

```
return 0;
}
```

上面的代码多数可以“望文知义”。结构体 `Point` 中定义了一个函数，函数名也叫 `Point`，但是没有返回值。这样的函数称为构造函数（ctor）。构造函数是在声明变量时调用的，例如，声明 `Point a, b(1,2)` 时，分别调用了 `Point()` 和 `Point(1,2)`。注意这个构造函数的两个参数后面都有“=0”字样，其中 0 为默认值。也就是说，如果没有指明这两个参数的值，就按 0 处理，因此 `Point()` 相当于 `Point(0,0)`。“`:x(x),y(y)`”则是一个简单的写法，表示“把成员变量 `x` 初始化为参数 `x`，成员变量 `y` 初始化为参数 `y`”。也可以写成：

```
Point(int x=0, int y=0) { this->x = x; this->y = y; }
```

这里的“`this`”是指向当前对象的指针。`this->x` 的意思是“当前对象的成员变量 `x`”，即 `(*this).x`。

提示 5-8: C++ 中的结构体可以有一个或多个构造函数，在声明变量时调用。

提示 5-9: C++ 中的函数（不只是构造函数）参数可以拥有默认值。

提示 5-10: 在 C++ 结构体的成员函数中，`this` 是指向当前对象的指针。

接下来为这个结构体定义了“加法”，并且在实现中用到构造函数。这样，就可以用 `a+b` 的形式计算两个结构体 `a` 和 `b` 的“和”了。

最后，定义这个结构体的流输出方式，然后就可以用 `cout << p` 来输出一个 `Point` 结构体 `p` 了。

5.1.5 模板

回顾第 4 章中介绍过的 `sum` 函数：

```
int sum(int* begin, int* end) {
    int *p = begin;
    int ans = 0;
    for(int *p = begin; p != end; p++)
        ans += *p;
    return ans;
}
```

这个函数没有错误，但比较局限——只能求整数数组的和，不能求 `double` 数组的和，更不能求 `Point` 数组的和。没关系，可以把这个函数改一下。

```
template<typename T>
T sum(T* begin, T* end) {
    T *p = begin;
    T ans = 0;
    for(T *p = begin; p != end; p++)
        ans = ans + *p;
}
```



```
return ans;
```

```
}
```

这样，就可以用 `sum` 函数给 `double` 数组和 `Point` 数组求和了。

```
int main() {
    double a[] = {1.1, 2.2, 3.3, 4.4};
    cout << sum(a, a+4) << "\n";
    Point b[] = { Point(1,2), Point(3,4), Point(5,6), Point(7,8) };
    cout << sum(b, b+4) << "\n";
    return 0;
}
```

细心的读者应该已发现了上述 `sum` 函数和第4章中写的有点不同：把“`ans += *p`”改成了“`ans = ans + *p`”。这样做的原因是 `Point` 结构体中只定义了“+”运算符，没有定义“+=”。结构体和类（`class`）也可以是带模板的。例如，上述 `Point` 结构体中的 `x` 和 `y` 是 `int` 型的，但有时需要的是 `double` 型的 `x` 和 `y`，“+”和“<<”的逻辑不变。可以用类似的写法把 `Point` 变成模板。

```
template <typename T>
struct Point {
    T x, y;
    Point(T x=0, T y=0):x(x),y(y) {}
};
```

然后把“+”和“<<”的代码也稍加改变：

```
template <typename T>
Point<T> operator + (const Point<T>& A, const Point<T>& B) {
    return Point<T>(A.x+B.x, A.y+B.y);
}
```

```
template <typename T>
ostream& operator << (ostream &out, const Point<T>& p) {
    out << "(" << p.x << ", " << p.y << ")";
    return out;
}
```

这样就可以同时使用 `int` 型和 `double` 型的 `Point` 了：

```
int main() {
    Point<int> a(1,2), b(3,4);
    Point<double> c(1.1,2.2), d(3.3,4.4);
    cout << a+b << " " << c+d << "\n";
    return 0;
}
```